

Unit - II

CS-402 ADA

GREEDY METHODS

prepared by:

Mr. Dayanand Yadav

Asst. prof.

CSE Department

Unit - 2

Greedy Method (Introduction) ①

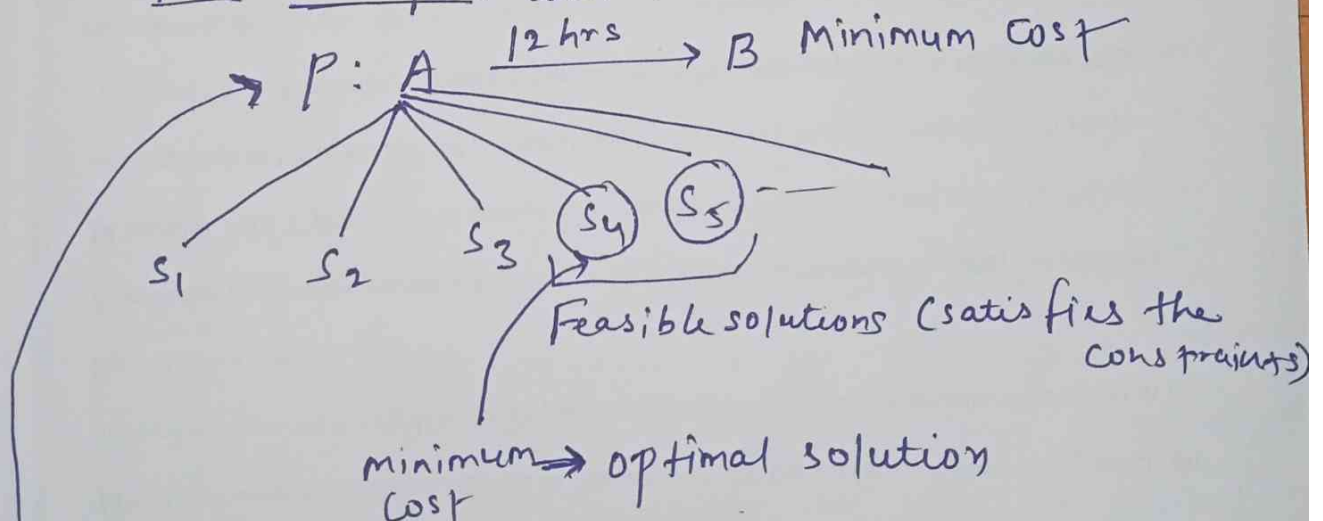
Just like other strategies like divide & conquer greedy method is also a strategy for solving a problem.

This method is used for solving optimization problems.

What ~~are~~ ^{are} optimization problems?

A problem which demands or requires either minimum result or maximum result (best possible outcome) ^(optimal solution) from a set of feasible options, while ~~adhering~~ ^{adhering} to specific constraints.

for example: constraint



for any problem there can be only one optimal solution.

& that kind of problem called optimization problem

Greedy Method (Introduction)

(1.1)

$$n = 5$$

a_1	a_2	a_3	a_4	a_5
1	2	3	4	5

Algorithm Greedy (a, n)

{ solution = 0; // Initialize the solution
for ($i = 1$ to n) // ~~inputs~~

do

{

$x = \text{select}(a);$

if Feasible(x) Then

{

 solution = solution + x ;

}

} return ~~optimal~~ solution;

Example: If you want a best car then how do you choose a best car?

Method 1: You should look at all the brands and all the models of car available in the world or in your city, then you can say I have checked all the cars and selected best car.

Method 2: Select a ^(Hyundai) brand $\rightarrow$ Select ⁽ⁱ¹⁰⁾ Model $\rightarrow$ Select feature & color $\rightarrow$ Then that is best car for you.

(Greedy Approach)

Greedy Method (Introduction) 1.2

Method 3: You are running a company
(Greedy Approach) and you want to hire people.
You given an ad & 1000 people
Came for interview then
You conduct Technical test,
Aptitude Test, personal Interview
and selects one person then
You say that this is best person.
You have got the optimal selection.

(2)

Example of Greedy Method

Optimal Merge pattern

Merging

A	B	C
3	5	3
8	9	5
12	11	8
20	16	9
<u>m</u>	<u>n</u>	11
(4)	(4)	12
		16
		20
		<u>(m+n)</u>
		(8)

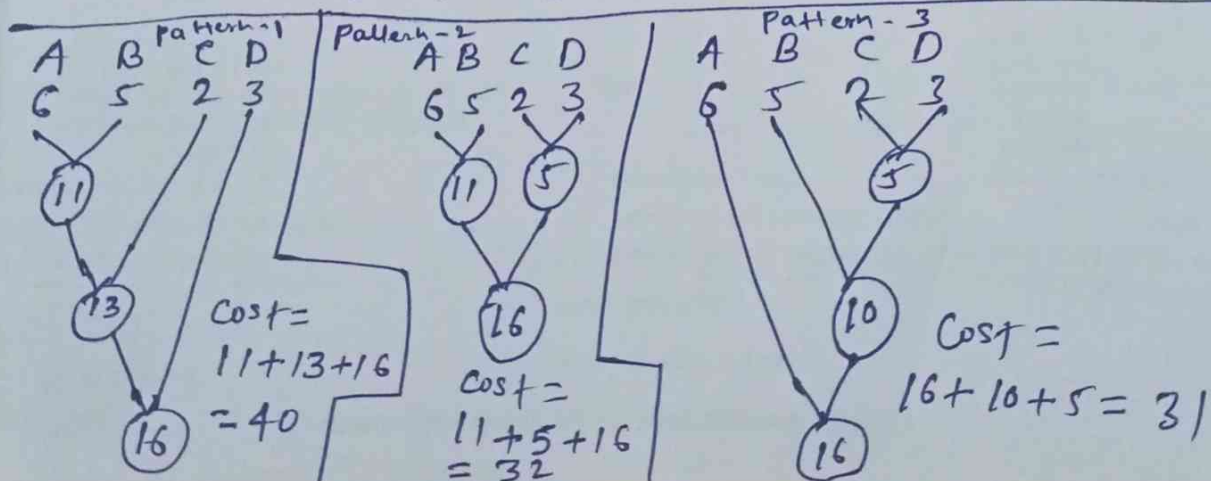
Given n number of sorted files, the task is to find the minimum computations done to reach the optimal merge pattern.

When two or more sorted files are to be merged altogether to form a single file, the minimum computations are done to reach this file are known as optimal Merge pattern.

Optimal Merge pattern

list $\rightarrow$ A B C D

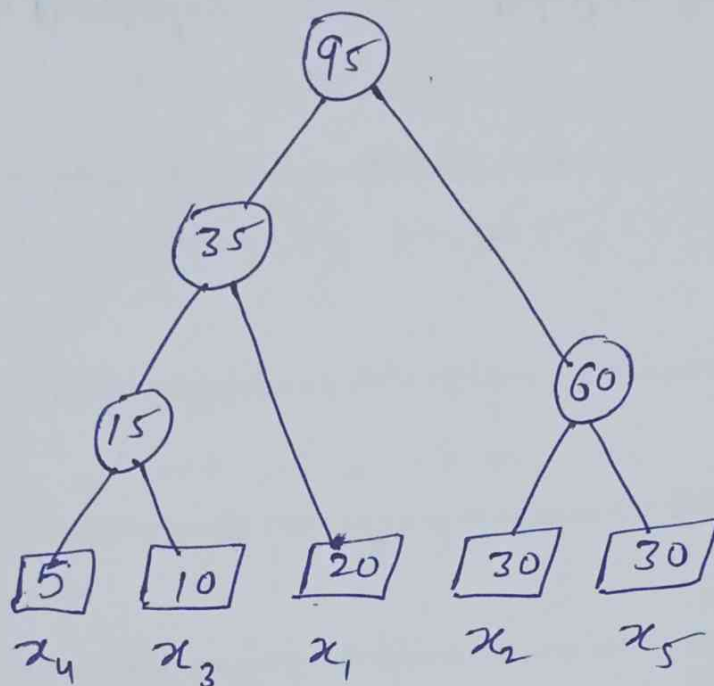
sizes $\rightarrow$ 6 5 2 3



Optimal Merge patterns

2.1

lists $\rightarrow x_1 \quad x_2 \quad x_3 \quad x_4 \quad x_5$
Sizes $\rightarrow 20 \quad 30 \quad 10 \quad 5 \quad 30$



(Binary ~~Tree~~ Merge Tree representing a merge pattern)

Total cost:

Method 1: $15 + 35 + 95 + 60 = 205$

Method 2: $3 \times 5 + 3 \times 10 + 2 \times 20 + 2 \times 30 + 2 \times 30 = 205$

Total cost = $\sum d_i \times x_i$
of merging

 $\leftarrow$ Weighted
external
path length
of the tree

This is used by Huffman coding

Optimal Merge Pattern

(2.2)

Tree algorithm to generate an optimal two way merge tree

```
treenode = record {  
    treenode *lchild;  
    treenode *rchild;  
    integer weight;  
};
```

1. Algorithm Tree(n)
2. // list is a global list of n single
3. // node binary trees ~~as~~ ~~there~~
4. {
5. for i = 1 to n-1 do
6. {
7. pt = new treenode; // Get a new tree node
8. (pt → lchild) = Least(list); // Merge two
9. (pt → rchild) = Least(list); // trees with
10. (pt → weight) = ((pt → lchild) → weight) // smallest length
11. + ((pt → rchild) → weight);
12. Insert(list, pt);
13. }
14. return Least(list); // Tree left in
15. } // list is the
16. } // merge tree

Applications of Optimal Merge patterns:

- ① External sorting ② DB query optimization ③ Data Compression

3

Huffman Coding

Definition: Huffman coding is a lossless data compression algorithm that assigns variable-length binary codes to characters based on how frequently they appear.

Message: B C C A B B D D A E C C B B A E D D C C
(20 characters)

Char	Count	Code	Message Size
A	3	001	$3 \times 3 = 9$
B	5	10	$5 \times 2 = 10$
C	6	11	$6 \times 2 = 12$
D	4	01	$4 \times 2 = 8$
E	2	000	$2 \times 3 = 6$

$5 \times 8 = 40$ bits

20 Characters

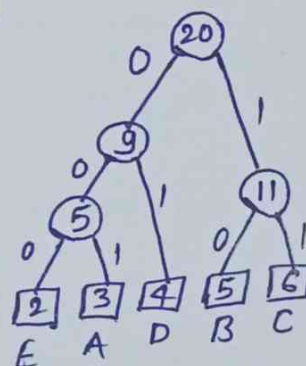
12 bit Codes

45 bits Message

8 bit ASCII Code

Message size = 45 bits

Tree/Table = 52 bits
97 bits



Huffman Tree

Compression % =

$$= \frac{97}{160} \times 100$$

$$\approx 61\%$$

This is Encoding process

61%

Huffman Coding

(3.1)

Decoding process

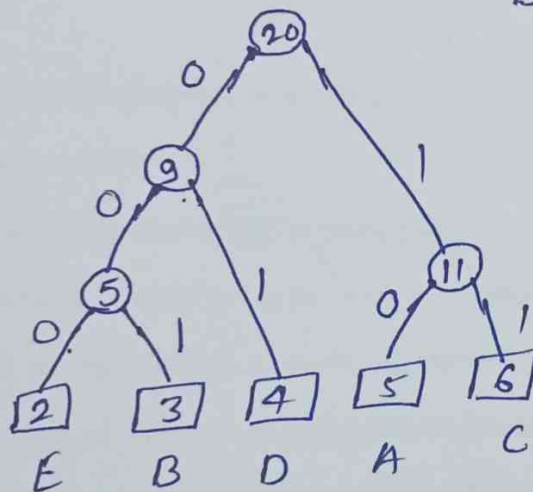
Message: B C C D A C C B D
 001 11 11 10 10 11 11 001 01

A B C C D E A A E
10 001 11 11 01 000 10 10 000

D A
01 10

Decoding

B C C D A - - - - -



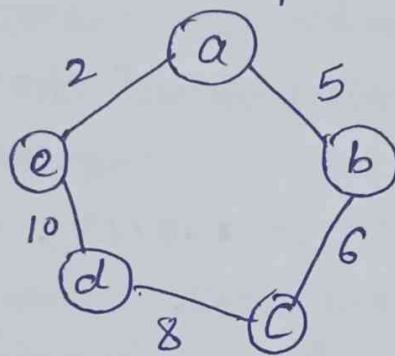
Applications of Huffman coding:

- ① File Compression
- ② Multimedia Codecs
- ③ HTTP Header Compression
- ④ Cryptography

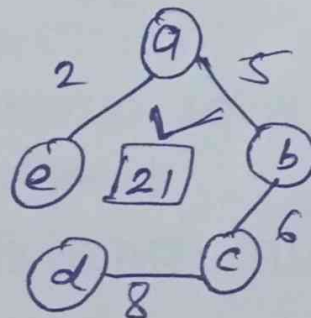
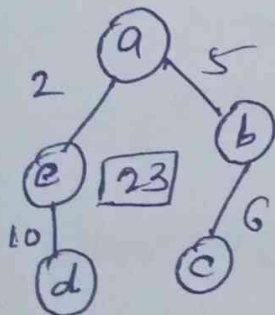
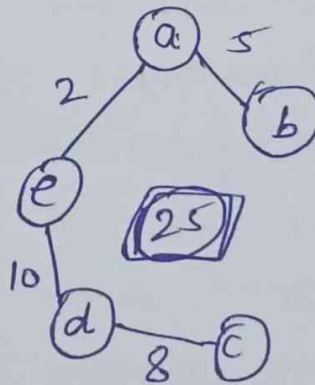
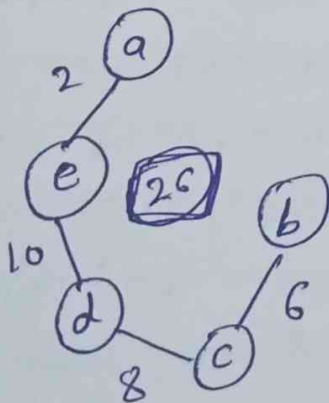
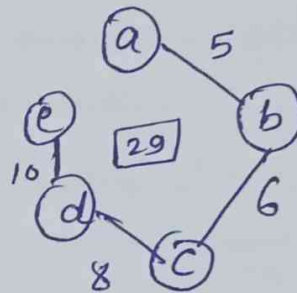
Minimum spanning Tree (MST) ④

A minimum spanning tree (MST) is defined as a spanning tree that has the minimum weight among all the possible spanning trees.

Graph



Spanning Tree



(MST)

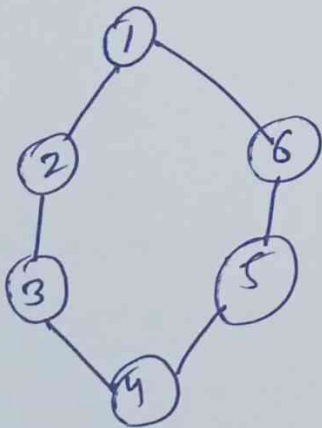
Minimum Cost spanning Tree

(4.1)

Prim's Algorithm:

A greedy method to obtain a minimum-cost spanning tree builds this tree edge-by-edge. The next edge to include is chosen according to some optimization criterion. The simplest such criterion is to choose an edge that results in a minimum increase in the sum of the cost of the edges so far included.

Graph

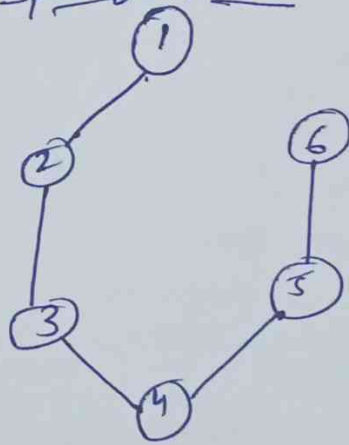


$$G = (V, E)$$

$$V = \{1, 2, 3, 4, 5, 6\}$$

$$E = \{(1,2), (2,3), (3,4), \dots\}$$

Spanning Tree



$$V' = |V| = n = 6$$

$$E' = |V| - 1 = 5$$

$$S \subseteq G$$

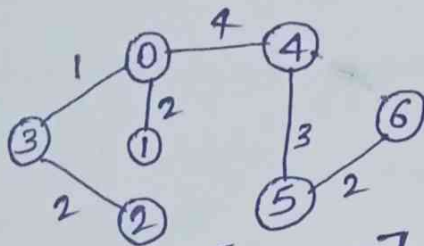
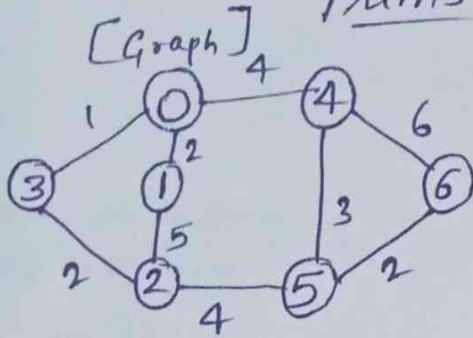
$$S = (V', E')$$

$$V' = V$$

$$|E'| = |V| - 1$$

Minimum spanning Tree (4.2)

Prim's Algorithm



MST [Minimum spanning Tree]

Initially

Parent [] Array

0	1	2	3	4	5	6

	0	1	2	3	4	5	6
0	0	2	0	1	4	0	0
1	2	0	5	0	0	0	0
2	0	5	0	2	0	0	0
3	1	0	2	0	0	3	6
4	4	0	0	0	3	0	2
5	0	0	4	0	6	2	0
6	0	0	0	0	0	2	0

Adjacency Matrix

Initially

MST [] Array

f	f	f	f	f	f	f
0	1	2	3	4	5	6

Initially

Key [] Array

∞	∞	∞	∞	∞	∞	∞
0	1	2	3	4	5	6

Algorithm

parent = [], key = [], mst = []

For each vertex v in graph

key[v] = int_max, mst[v] = f

key[0] = 0, parent = -1

for each vertex v in graph

→ { K = min key[v] && MST[v] = f

mst[K] = t (true)

For each vertex v in graph

if (graph[K][v] && mst[v] is false && graph[K][v] < key[v])

key[v] = graph[K][v]

parent[v] = K

}←

finally

Parent	0	1	2	3	4	5	6
	-1	0	3	0	0	4	5

finally

MST	0	1	2	3	4	5	6
	T	T	T	T	T	T	T

finally

Key	0	1	2	3	4	5	6
	0	2	2	1	4	3	2

Minimum spanning Tree

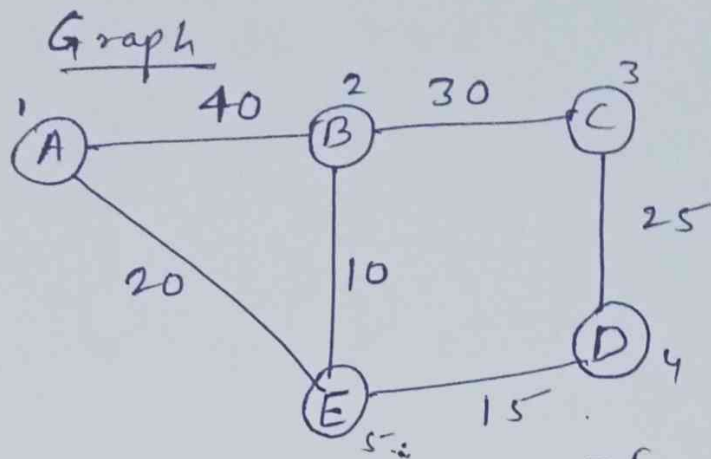
(4.3)

Kruskal's Algorithm

Kruskal's algorithm is a greedy algorithm used to find the minimum spanning Tree (MST). It works by sorting all edges by weight and adding the smallest edge one by one ensuring that no cycle are formed until all vertices are connected.

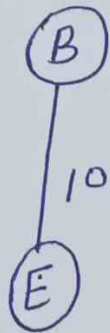
Kruskal Algorithm

(4.4)

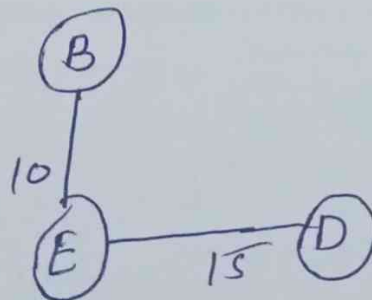


	AB	AE	BC	BE	CD	DE
Array →	40	20	30	10	25	15
Edges						
index	1	2	3	4	5	6

Step 1 → Pick B-E weight → 10 (Minimum)



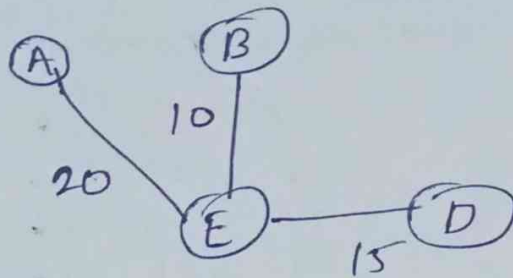
Step 2: Pick E-D with weight - 15
next Minimum



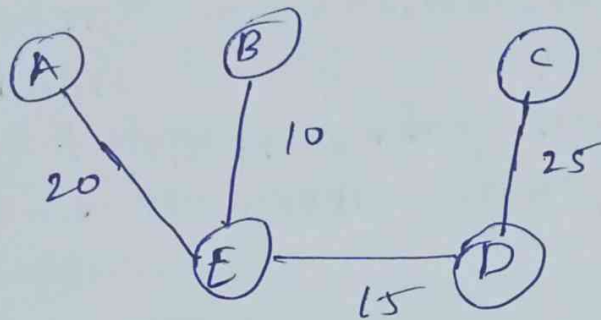
Kruskal's Algorithm

4.5

step 3: pick AD. weight 20



step 4: pick DE weight 25



Minimum cost spanning tree

$$\underline{\text{cost}} = \underline{70}$$

Kruskal's Algorithm

(4.6)

Algorithm Kruskal (E, cost, n, t)

// E is the set of edges in G . G has n
// vertices. $\text{cost}[u,v]$ is the cost of
// edge (u,v) . t is the set of edges in
// the minimum-cost spanning tree. the
// final cost returned.

{

construct a heap out of the edge cost
using Heapify;

for $i = 1$ to n do $\text{parent}[i] = -1$;

// Each vertex is in a different set.

$i = 0$; $\text{mincost} = 0.0$;

while $((i \leq n-1) \text{ and } (\text{heap not empty}))$

do

{

Delete a minimum cost edge (u,v)
from the heap and reheapify using Adjust;

$j = \text{find}(u)$; $k = \text{find}(v)$;

if $(j \neq k)$ then

{

$i = i + 1$;

$t[i,1] = u$; $t[i,2] = v$;

$\text{mincost} = \text{mincost} + \text{cost}[u,v]$;

Union (j, k) ;

}

} if $(i \neq n-1)$ then write("No spanning tree");
else return mincost ; }

~~HT~~ Min heap Insertion (4.7)

To get the edge with smallest weight

// function to insert into min-heap

```
void insertMinHeap(int item)
```

```
{
```

```
    heapSize++;
```

```
    int i = heapSize; // get no. of Edges weights
```

```
    heap[i] = item; // place the new item  
                    // @ the end
```

// Sorting (Bubble sort)

// TO get smallest element (weight) on top.

```
while (i > 1 && heap[i/2] > heap[i])
```

```
{
```

```
    int temp = heap[i]
```

```
    heap[i] = heap[i/2];
```

```
    heap[i/2] = temp;
```

```
    i = i/2;
```

```
}
```

```
}
```


Kruskal's Algorithm

4.8

Algorithm Adjust for Min-heap

Algorithm AdjustMin(a, i, n)

// Parent = i

// lchild = $2 * i$

// rchild = $2 * i + 1$

{

$j = 2i$;

 item = $a[i]$;

 while($j \leq n$) do

 {

 if (($j < n$) and ($a[j] > a[j+1]$))

 then

$j = j + 1$;

 if (item $\leq a[j]$) then break;

$a[j/2] = a[j]$;

$j = 2*j$;

 }

$a[j/2] = \text{item}$;

}

Kruskal algorithm

(49)

Algorithm Delete for Min heap

Algorithm DelMin(a, n, x)

// Delete the minimum from the heap
 $a[1:n]$ and store it in x .

{

if ($n = 0$) then

{

write ("heap is empty");

return false;

}

$x = a[1];$

$a[1] = a[n];$

AdjustMin($a, 1, n-1$);

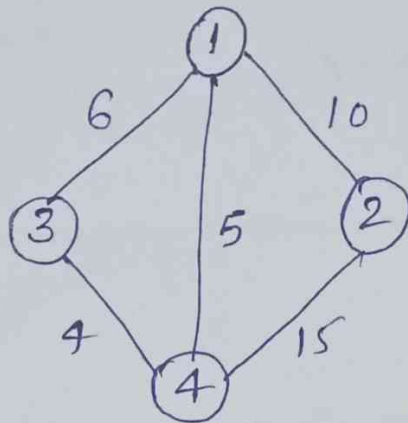
return ~~true~~
 x ;

}

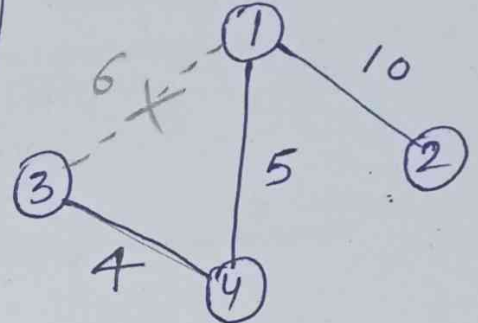
Kruskal's Algorithm

4.10

Graph

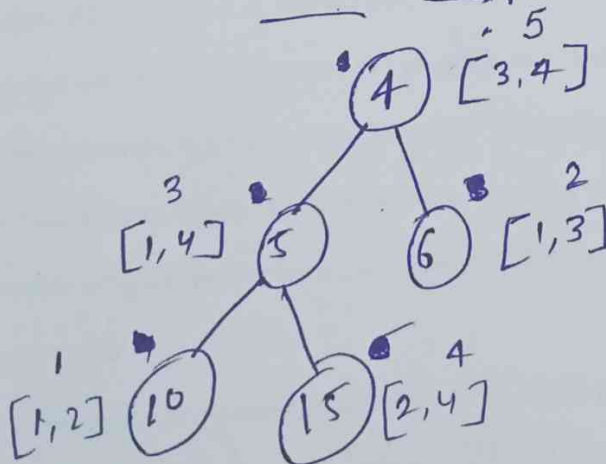


Parent[i] = -1

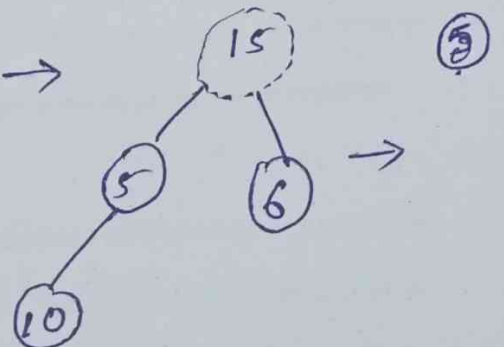


Edges	(u, v)	$(1, 2)$	$(1, 3)$	$(1, 4)$	$(2, 4)$	$(3, 4)$
Weight		10	6	5	15	4
index		1	2	3	4	5

Min heap



Minimum
pick weight = 4
and Adjust heap
& heapify()



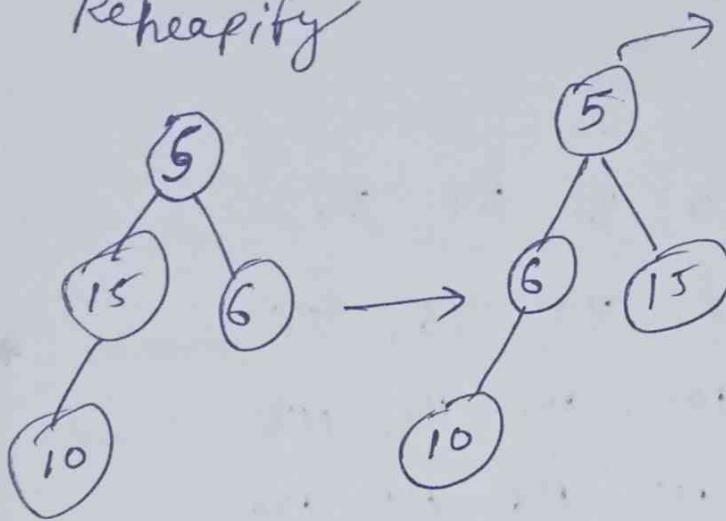
heapsize = 5

heapsize = 4

Min $w = 4$ $(u, v) = (3, 4)$

(4.11)

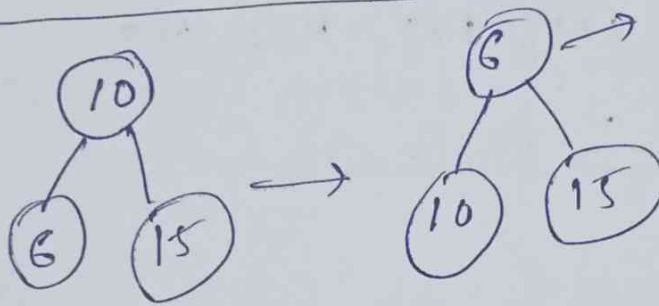
Reheapify



Minimum
pick weight = 5

~~(u, v, w)~~
(u, v, w)
(1, 4, 5)

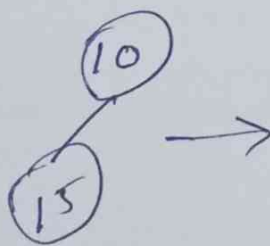
No cycle detected



Pick minimum
weight = 6

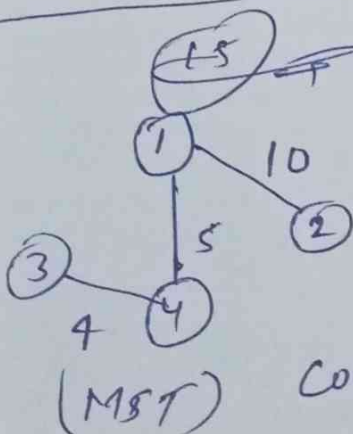
(u, v, w)
(1, 3, 6)

cycle detect, so
discard edge(1, 3, 6)



pick minimum weight = 10
(u, v, w) $\Rightarrow$ (1, 2, 10)

No cycle detected



Now all vertex are
covered so no need to
further searching

(MST) Cost of MST = 19

Prim's & Kruskal's Algorithm Comparison

4-12

Feature	Kruskal's Algorithm	Prim's Algorithm
Strategy	Edge based	Vertex based
Data structure	Union-Find	Priority Queue
Best for	sparse graphs	Dense graphs
Complexity	$O(E \log E)$ $O(n \log n)$	$O(V \log V)$ $O(m \log n)$

Applications of Prim's & Kruskal's Algorithms:

- ① Telecommunications
- ② Electrical grids
- ③ Computer networks
- ④ Roads & Railways
- ⑤ Cluster Analysis
- ⑥ Pathfinding
- ⑦ Data Analysis
- ⑧ Approximation Algorithms.

(5)

Knapsack Problem

Greedy Method

The Knapsack problem is a Combinatorial Optimization challenge where you must select the most valuable combination of ~~of~~ "items" to fit into a container with a fixed ~~size~~ weight limit.

Unlike the minimum spanning Tree (MST) from your previous topic - which uses a "Greedy" strategy (always picking the smallest edge) - the standard 0/1 Knapsack Problem can not be solved with simply greedy approach. It usually requires Dynamic programming.

But Fractional Knapsack Problem can be solved with simply Greedy Method.

- 0/1 Knapsack → Items must be taken entirely or left (0-1)
- Fractional Knapsack → Items can be broken down, allowing fractions of items to be taken.

(5.1)

Date: / / Page no:

thoq)

Knapsack problem

Greedy algorithm strategy:

The core idea is to prioritize items that offer the most ^{value} for every unit of weight. (profit)

(1) Calculate profit to weight Ratio

$$\frac{p}{w}$$

(2) Sort item and arrange them in descending order based on their Ratio

(3) Fill Knapsack:

- (i) Iterate through the sorted list.
- (ii) If item's weight fits ~~com~~ completely within the remaining capacity, add its full value. ^x
- (iii) If an item is heavier than the remaining capacity, take a fraction of it to fill the ^{x1} Knapsack exactly.

Time complexity: $O(n \log n)$ due to ~~so~~ sorting and calculating ratio and final iteration both take $O(n)$ ¹⁵

Space complexity: $O(n)$ ~~to~~ to store n items

Knapsack problem

(5.2)

Fractional Knapsack (Greedy Method)

$n=7$ Objects: $0 \rightarrow 1 \ 2 \ 3 \ 4 \ 5 \ 6 \ 7$

$m=15$ Profits: $P \rightarrow 10 \ 5 \ 15 \ 7 \ 6 \ 18 \ 3$

Weights: $w \rightarrow 2 \ 3 \ 5 \ 7 \ 1 \ 4 \ 1$

$\frac{P}{w} \rightarrow 5 \ 1.3 \ 3 \ 1 \ 6 \ 4.5 \ 3$



x_i (x_1 x_2 x_3 x_4 x_5 x_6 x_7)
 $\begin{pmatrix} s_2 & s_2 & s_4 & 0 & s_1 & s_3 & s_5 \\ 1 & 2/3 & 1 & 0 & 1 & 1 & 1 \end{pmatrix}$

$$0 \leq x \leq 1$$

Object may be vegetables/ grain

Constraint
$\sum x_i w_i \leq m$
Objective
$\max \sum x_i P_i$



$$\begin{aligned} 15 - 1 &= 14 \\ 14 - 2 &= 12 \\ 12 - 4 &= 8 \\ 8 - 5 &= 3 \\ 3 - 1 &= 2 \\ 2 - 2 &= 0 \end{aligned}$$

Verify ~~weight~~

$$\sum_{i=1}^n x_i w_i = (1 \times 2) + (2/3 \times 3) + (1 \times 5) + (0 \times 7) + (1 \times 1) + (1 \times 4) + (1 \times 1)$$

$$\text{Total weight } \sum x_i w_i = 2 + 2 + 5 + 0 + 1 + 4 + 1 = 15$$

$$\text{Total profit } \sum x_i P_i = (1 \times 10) + (2/3 \times 5) + (1 \times 15) + (0 \times 7) + (1 \times 6) + (1 \times 18) + (1 \times 3)$$

$$\sum x_i P_i = 10 + (0.66 \times 5) + 15 + 0 + 6 + 18 + 3 = 55.33 \Rightarrow \text{Total profit}$$

Job sequencing with deadlines (6)

Greedy Method

Date: / / Page no:

Job sequencing with deadlines is a greedy algorithm problem where you aim to schedule a set of jobs to maximize total profit.

Each job has specific deadline and an associated profit and it's assumed that every job takes exactly one unit of time to complete.

The Greedy algorithm steps:
To find optimal schedule

- ① Sort jobs by profit
- ② Identify maximum deadline
- ③ Allocate slots

Job sequencing with deadlines (6.1)

Greedy Method

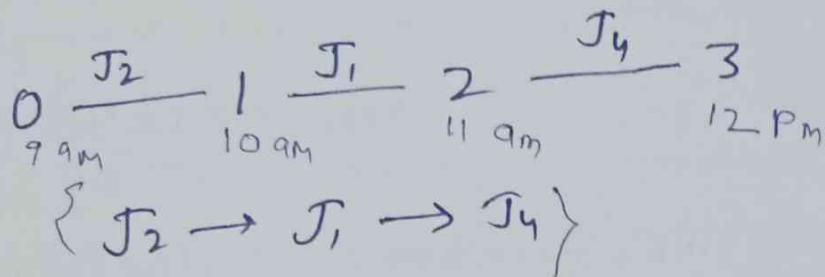
$n=5$

Example - 1

Jobs	J_1	J_2	J_3	J_4	J_5
profits	20	15	10	5	1
Deadlines	2	2	1	3	3

$$20 + 15 + 05 = 40$$

$$\{J_1, J_2, J_4\}$$



$$J_1 \rightarrow J_2 \rightarrow J_4$$

Job Consider	slot assign	solution	profit
—	—	$\emptyset$	0
J_1	$[1, 2]$	J_1	20
J_2	$[0, 1] [1, 2]$	J_1, J_2	$20 + 15$
$J_3 \times$	$[0, 1] [1, 2]$	J_1, J_2	$20 + 15$
J_4	$[0, 1] [1, 2] [2, 3]$	J_1, J_2, J_4	$20 + 15 + 5$
$J_5 \times$	—	—	40

Total profit = 40

Job sequencing with deadlines (6.2)

$n=7$

Example - 2

Jobs:	J_1	J_2	J_3	J_4	J_5	J_6	J_7
Profits:	35	30	25	20	15	12	5
Deadlines:	3	4	4	2	3	1	2
					x	x	x

$$0 \frac{J_4}{20} \quad 1 \frac{J_3}{25} \quad 2 \frac{J_1}{35} \quad 3 \frac{J_2}{30} \quad 4 = 110$$

Job Consider	slot Assign	solution	profit
—	—	ϕ	0
J_1	[2,3]	J_1	35
J_2	[2,3] [3,4]	J_1, J_2	35+30
J_3	[2,3] [3,4] [1,2]	J_1, J_2, J_3	35+30+25
J_4	[2,3] [3,4] [1,2] [0,1]	J_1, J_2, J_3, J_4	35+30+25+20

Total profit = 110

Single source shortest path (SSSP) algorithm

(7)

Dijkstra's Algorithm

A single source path (SSSP) algorithm finds the minimum distance from a single starting vertex (the source) to all other vertices in weighted graph.

Dijkstra's Algorithm

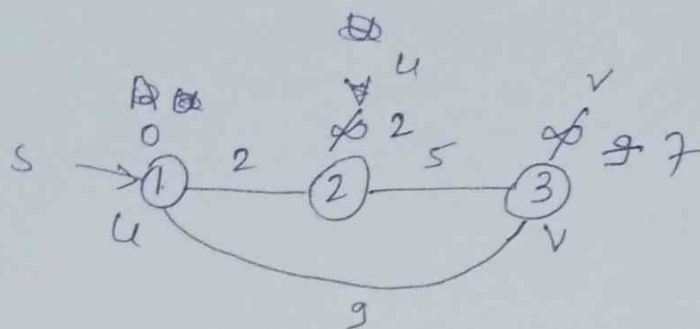
Dijkstra's algorithm is one of the very famous greedy algorithms.

It is used for solving the single source shortest path problem which gives the shortest paths from one particular source node to all the other nodes of the given graph.

Dijkstra's is the most common choice for real-world mapping (like Google Maps) because it is fast & efficient.

Dijkstra's Algorithm

7.1



Source vertex = ①

Relaxation

1-2 $d(u) + c(u,v) < d(v)$

$$d(v) = d(u) + c(u,v)$$

$$d(u) + c(u,v) < \infty$$

$$0 + 2 < \infty$$

$$d(v) = d(u) + c(u,v)$$

$$d(v) = 0 + 2$$

	1	2	3
1	0	∞	∞
1, 2		②	∞
1, 3		②	9
1, 2, 3		②	⑦ ←

1, 2, 3 → ⑦

1-3 $d(u) + c(u,v) < \infty$

$$0 + 9 < \infty$$

$$d(v) = d(u) + c(u,v)$$

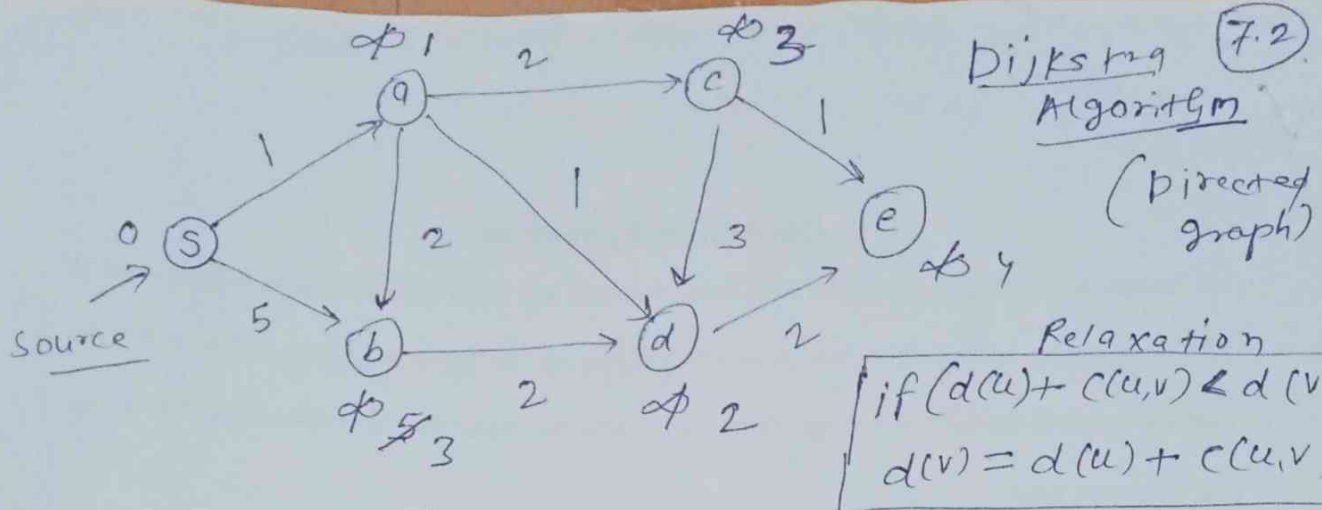
$$= 0 + 9$$

1-2-3, $d(u) + c(u,v) < \infty$

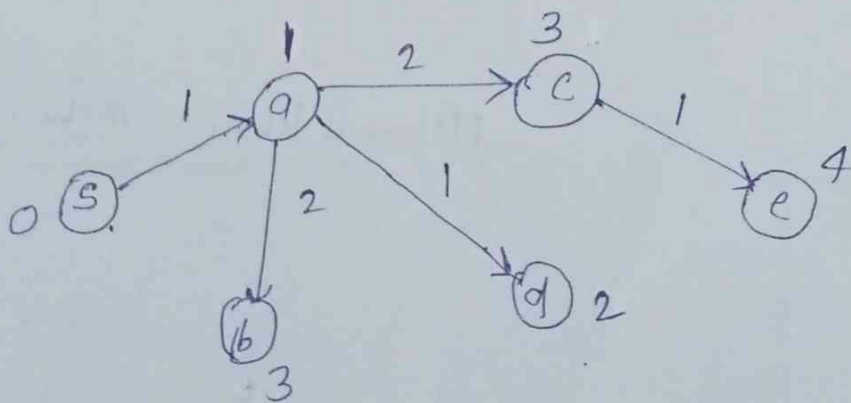
$$2 + 5 < 9$$

$$d(v) = d(u) + c(u,v)$$

$$2 + 5 = 7$$



selected/ visited vertices	s	a	b	c	d	e
s	0	∞	∞	∞	∞	∞
s, a	0	1	5	∞	∞	∞
s, a s, a, d	0	1	3	3	2	∞
s, a, d, b	0	1	3	3	2	4
s, a, d, b, c	0	1	3	3	2	4
s, a, d, b, c, e	0	1	3	3	2	4



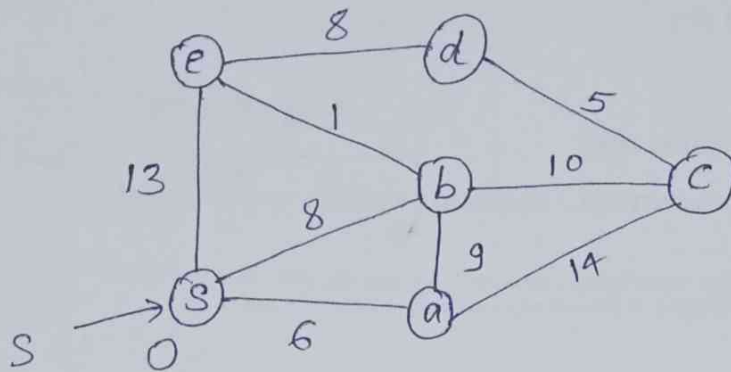
(SSSP)
Single source shortest path

Total Cost = $0 + 1 + 3 + 4 + 2 + 3$
= 13

Single source shortest path (SSSP)

Dijkstra's Algorithm (7.3)

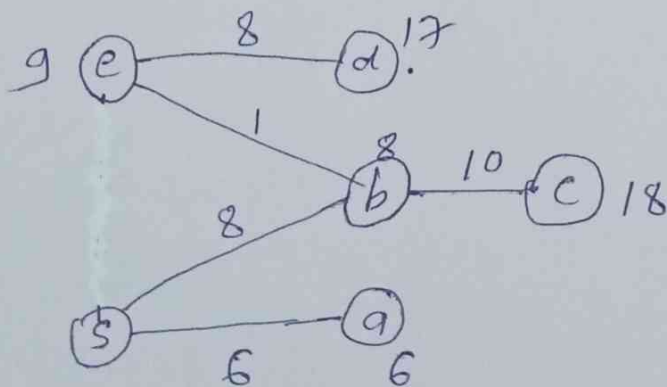
(Undirected Graph)



Relaxation
if $d(u) + c(u,v) < d(v)$

$$d(v) = d(u) + c(u,v)$$

s	a	b	c	d	e	Selected vertex
0	∞	∞	∞	∞	∞	s
-	6	8	∞	∞	13	s, a
-	-	8	18	∞	13	s, a, b
-	-	-	18	∞	9	s, a, b, e
-	-	-	18	17	-	s, a, b, e, d
-	-	-	18	-	-	s, a, b, e, d, c



Single source shortest path (SSSP)

$$\text{Total cost} = 0 + 6 + 8 + 18 + 17 + 9 = 58$$

Thank You